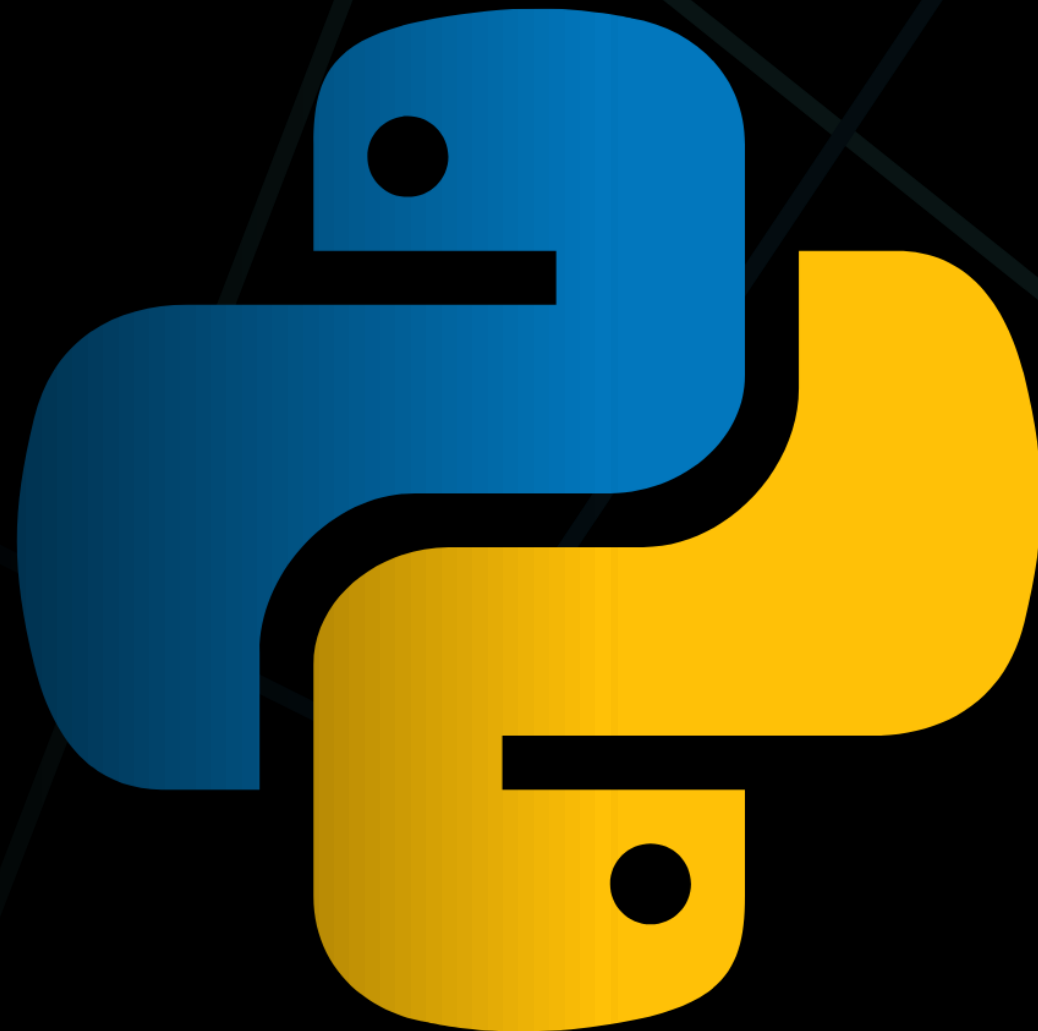




O que são funções em python

Programação em python



Fundamento Técnico: o conceito de Função

—

Uma função em Python é um bloco independente de código criado para executar uma tarefa específica. Ela é projetada para receber valores, processar dados e retornar resultados, de forma organizada, reutilizável e modular.

Em termos técnicos, as funções evitam repetição de código e aumentam a manutenibilidade dos sistemas, sendo um dos fundamentos da engenharia de software estruturada.

Fundamento Técnico: o conceito de Função

Estrutura sintática básica:

```
def nome_da_funcao():  
    # bloco de código
```

def → palavra reservada que indica a criação de uma função;

nome_da_funcao → identificador (segue as regras de nomeação vistas na PEP 8);

() → define o espaço para parâmetros de entrada;

: → marca o início do bloco indentado.

Explicação conceitual simples

—

Pense em uma função como uma máquina de tarefas:

- Ela recebe algo (dados, valores ou informações);
- Realiza um processamento interno;
- E retorna um resultado.

Em um ambiente corporativo, funções são usadas em cadastros automáticos, cálculos de relatórios, validação de formulários, controle logístico e sistemas de auditoria.

The background is a dark blue gradient with glowing orange and yellow lines that curve across the frame. Scattered throughout are binary digits (0s and 1s) in various colors, including white, yellow, and blue, some appearing to float or move. A thin horizontal orange line is positioned behind the text.

Diferença entre: código linear e código funcional

```
1 # Código linear (sem função):
2 print("Iniciando cálculo de comissão...")
3 venda = 1500
4 comissao = venda * 0.1
5 print("Comissão calculada:", comissao)
6
7 print("Iniciando cálculo de comissão...")
8 venda = 2300
9 comissao = venda * 0.1
10 print("Comissão calculada:", comissao)
11
12 #Esse tipo de código repete blocos iguais, o que aumenta o risco de
    erro e dificulta a manutenção.
```

```
1 # Código funcional (com função):
2 def calcular_comissao(valor_venda):
3     comissao = valor_venda * 0.1
4     print("Comissão calculada:", comissao)
5
6 calcular_comissao(1500)
7 calcular_comissao(2300)
8
9 # O uso de função transforma o processo em estrutura modular,
  permitindo que o mesmo bloco seja executado várias vezes com
  diferentes valores de entrada. Esse é o princípio da reutilização de
  código, base da programação profissional.
```


Cálculo de Salário Líquido

ArquivoEditorSeleçãoVerAcessarExecutarTerminalAjuda

Q AulasPython [Administrador]

aulas.py x

aulas.py > ...

1

def calcular_salario_liquido(salario_bruto):

2

if salario_bruto <= 1320:

3

desconto_inss = salario_bruto * 0.08

4

elif salario_bruto <= 2571:

5

desconto_inss = salario_bruto * 0.09

6

else:

7

desconto_inss = salario_bruto * 0.11

8

9

salario_liquido = salario_bruto - desconto_inss

10

return salario_liquido

11

12

salario = float(input("Digite o salário bruto: "))

13

resultado = calcular_salario_liquido(salario)

14

print("Salário líquido:", resultado)

15

Ln 15, Col 1

Espaços: 4

UTF-8

CRLF

{ } Python

Finish Setup

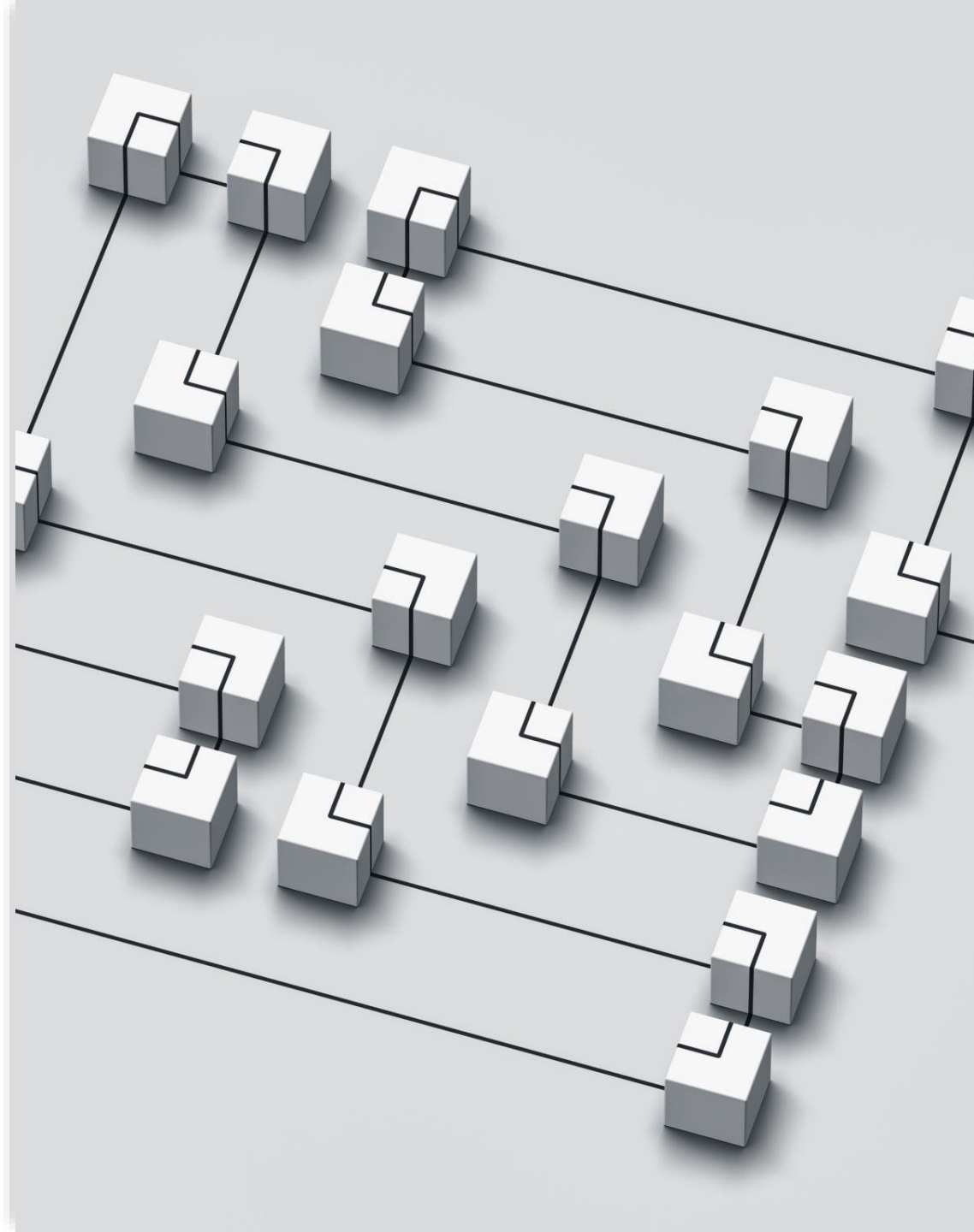
3.13.2



```
aulas.py x
aulas.py
1 # Define uma função chamada calcular_salario_liquido que recebe o salário bruto como parâmetro
2 def calcular_salario_liquido(salario_bruto):
3
4     # Verifica se o salário é menor ou igual a 1320
5     if salario_bruto <= 1320:
6         # Calcula o desconto de 8% sobre o salário bruto
7         desconto_inss = salario_bruto * 0.08
8
9     # Verifica se o salário é menor ou igual a 2571 (segunda faixa)
10    elif salario_bruto <= 2571:
11        # Calcula o desconto de 9% sobre o salário bruto
12        desconto_inss = salario_bruto * 0.09
13
14    # Caso o salário esteja acima das faixas anteriores
15    else:
16        # Calcula o desconto de 11% sobre o salário bruto
17        desconto_inss = salario_bruto * 0.11
18
19    # Calcula o salário líquido subtraindo o desconto do salário bruto
20    salario_liquido = salario_bruto - desconto_inss
21
22    # Retorna o valor final calculado para quem chamou a função
23    return salario_liquido
24
25
26 # Solicita ao usuário que digite o salário bruto e converte o valor para número decimal
27 salario = float(input("Digite o salário bruto: "))
28
29 # Chama a função passando o salário digitado e armazena o resultado
30 resultado = calcular_salario_liquido(salario)
31
32 # Exibe o salário líquido calculado
33 print("Salário líquido:", resultado)
34
```

Análise técnica:

- A função recebe um parâmetro numérico (salario_bruto);
- Implementa regras de negócio com estrutura condicional;
- Realiza processamento interno antes do retorno;
- Utiliza return para devolver o valor calculado;
- Representa um padrão comum em sistemas de folha de pagamento;
- Permite reutilização mensal sem reescrita da lógica.





Sistema de Autenticação

ArquivoEditorSeleçãoVerAcessarExecutarTerminalAjuda

AulasPython [Administrador]

aulas.py x

aulas.py > ...

1def autenticar_usuario(usuario, senha):

2 usuario_padrao = "financeiro"

3 senha_padrao = "Empresa@2026"

4

5 if usuario.strip() == usuario_padrao and senha == senha_padrao:

6 return True

7 else:

8 return False

9

10 usuario = input("Usuário: ")

11 senha = input("Senha: ")

12

13 if autenticar_usuario(usuario, senha):

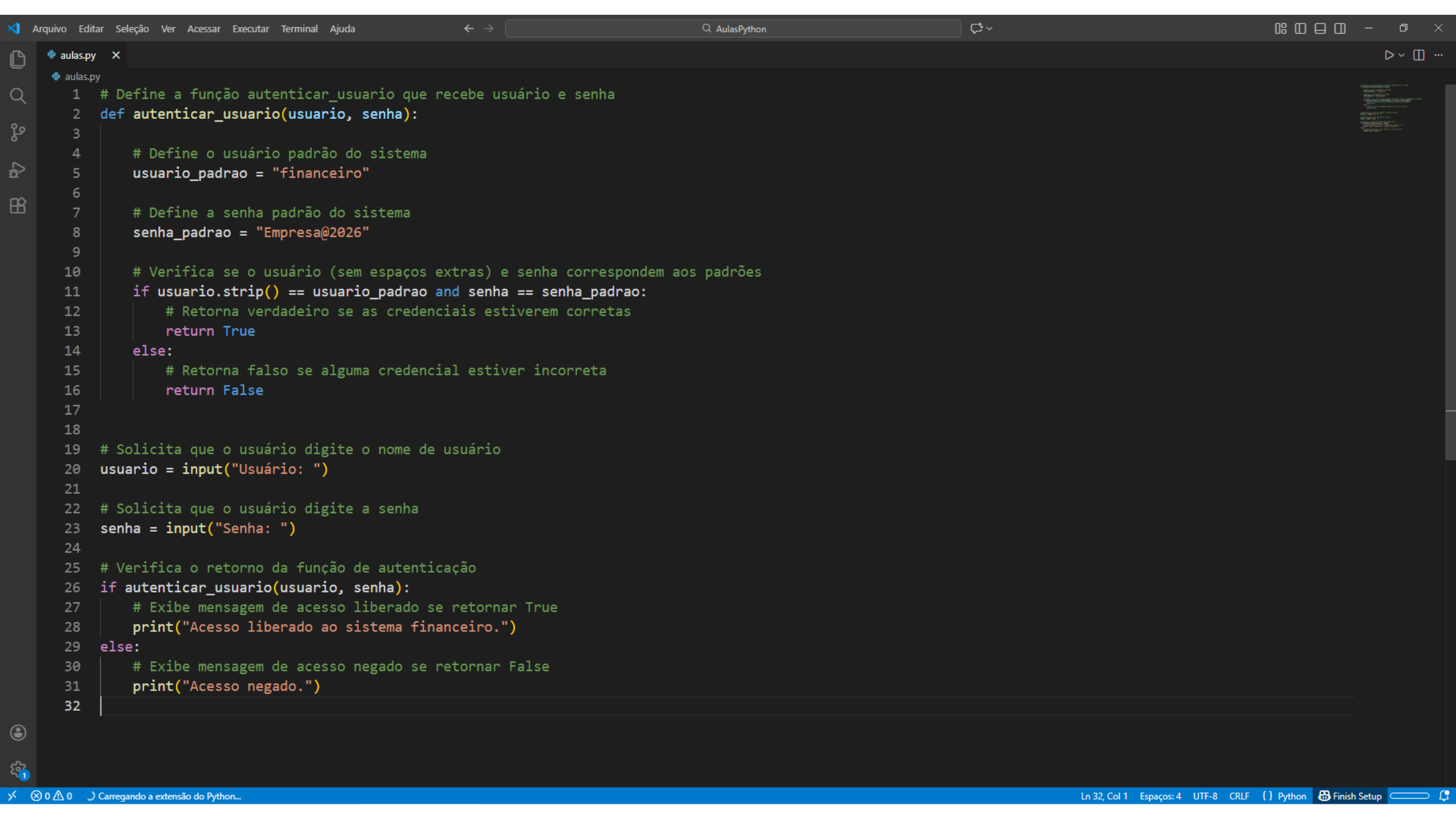
14 print("Acesso liberado ao sistema financeiro.")

15 else:

16 print("Acesso negado.")

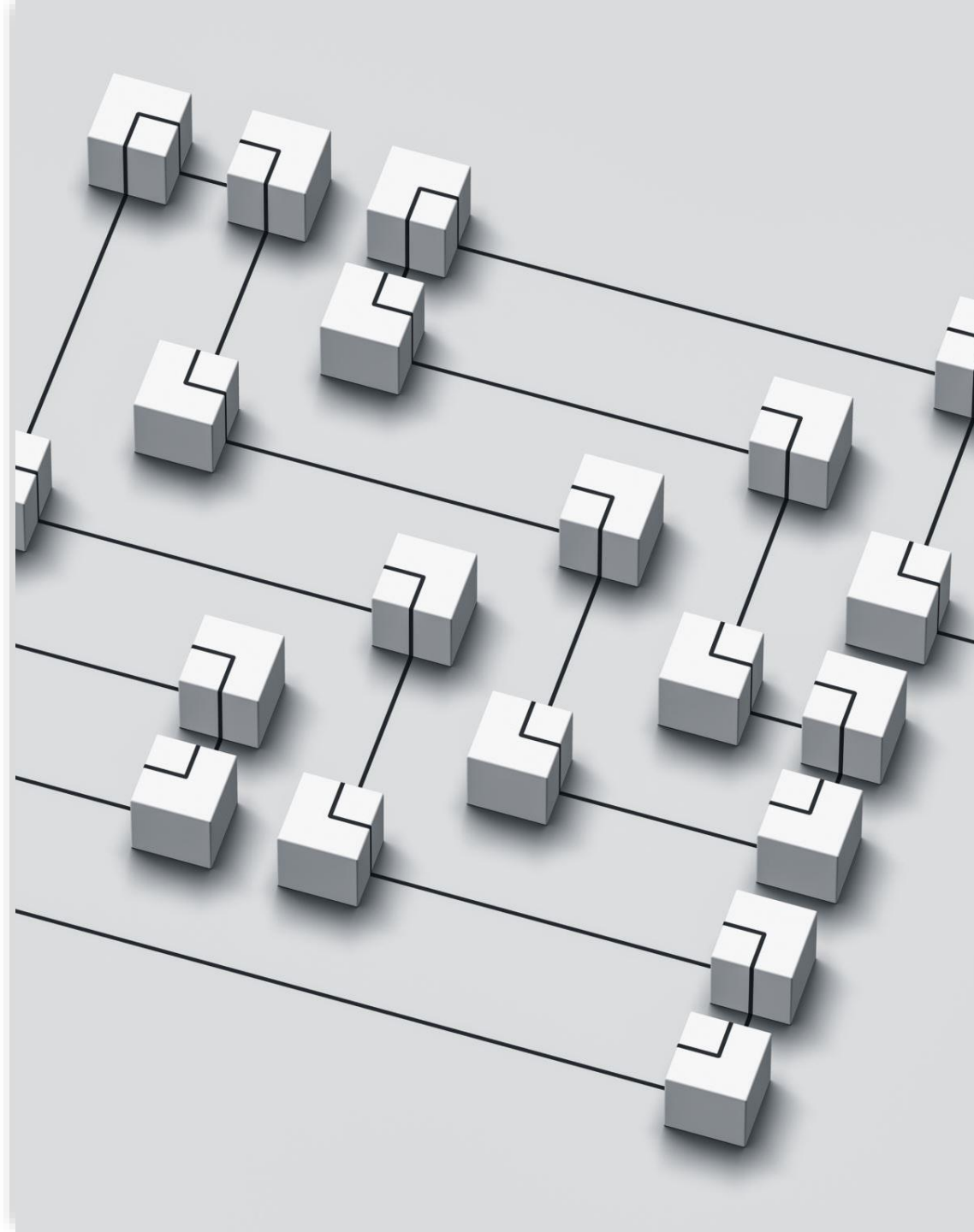
17

Ln 17, Col 1Espaços: 4UTF-8CRLFPythonFinish Setup3.13.2



Análise técnica:

- A função recebe dois parâmetros (usuario e senha);
- Utiliza `.strip()` para eliminar espaços indevidos;
- Integra operador lógico `and` para dupla verificação;
- Retorna valor booleano (True ou False);
- Representa modelo simplificado de autenticação corporativa;
- Demonstra separação clara entre validação e exibição.





Controle de Estoque



Arquivo Editar Seleção Ver Acessar Executar Terminal Ajuda

Q AulasPython [Administrador]

aulas.py x

aulas.py > ...

1 def atualizar_estoque(estoque_atual, quantidade_vendida):

2 if quantidade_vendida > estoque_atual:

3 return "Estoque insuficiente."

4

5 novo_estoque = estoque_atual - quantidade_vendida

6 return novo_estoque

7

8 estoque = 50

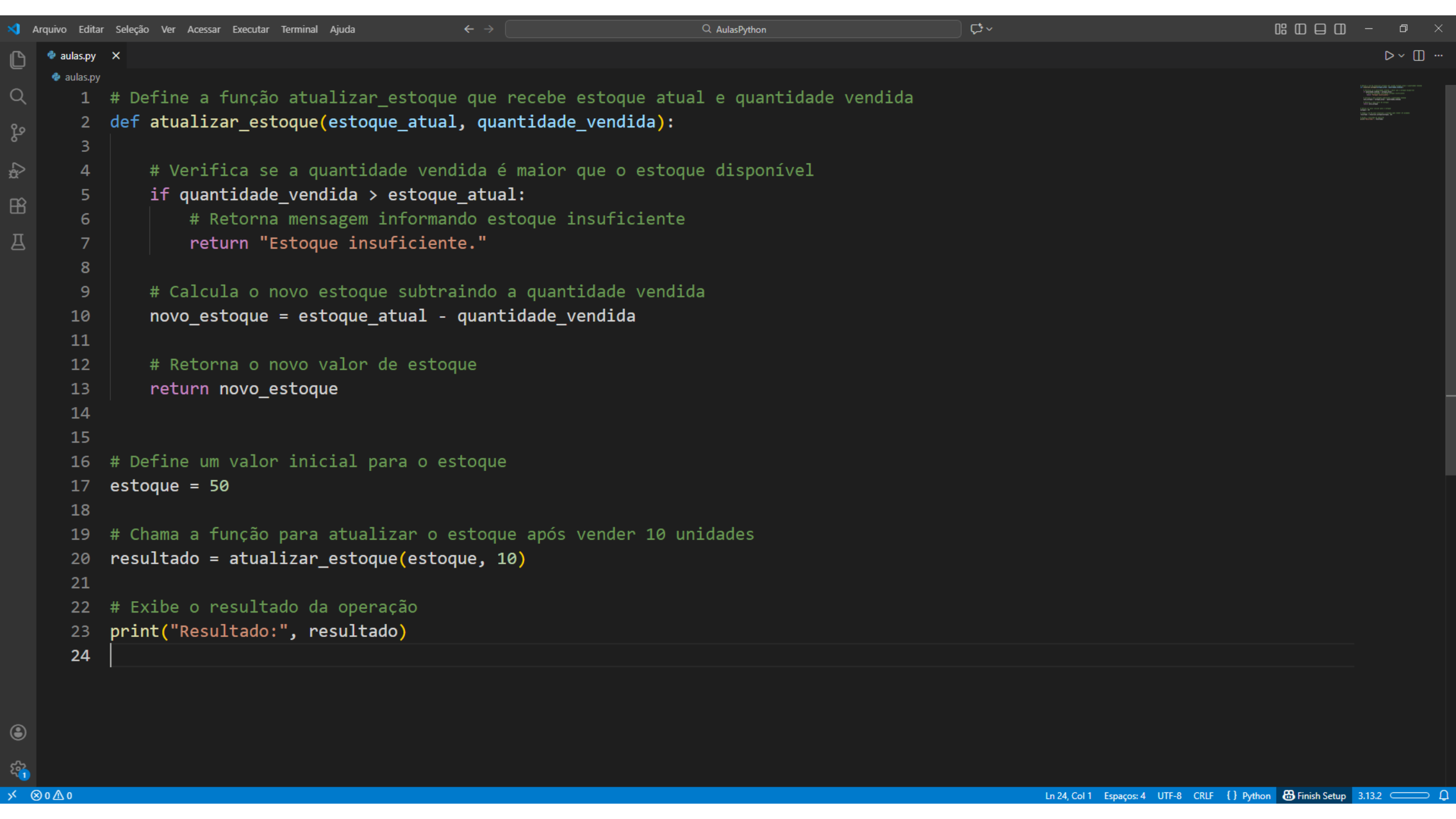
9 resultado = atualizar_estoque(estoque, 10)

10

11 print("Resultado:", resultado)

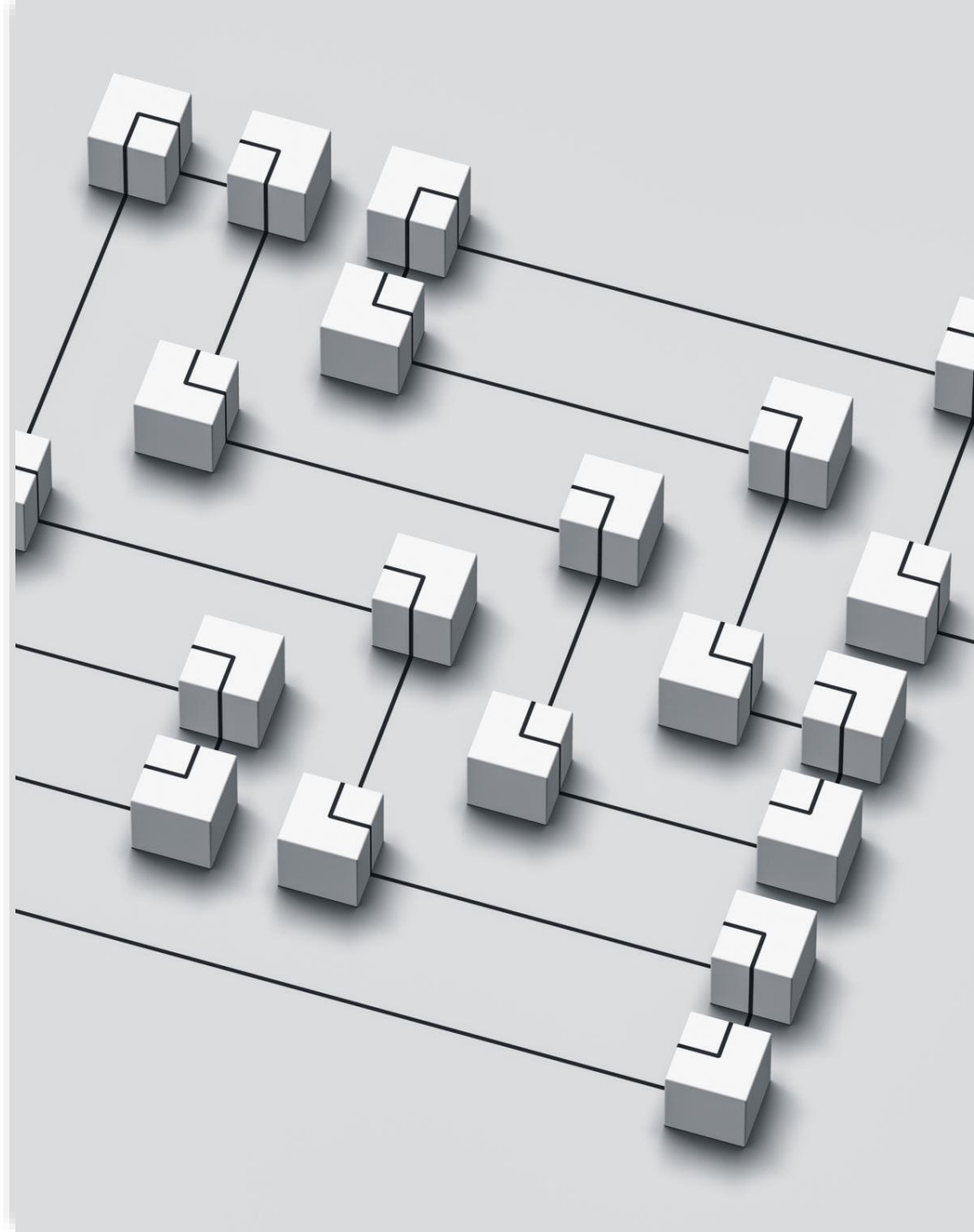
12

Ln 12, Col 1 Espaços: 4 UTF-8 CRLF Python Finish Setup 3.13.2



Análise técnica:

- A função recebe dois parâmetros numéricos;
- Realiza verificação preventiva antes da operação;
- Implementa controle lógico de integridade de dados;
- Retorna mensagem de erro ou valor atualizado;
- Simula comportamento típico de sistema ERP;
- Evita inconsistência no banco de dados.





Funções Interconectadas (Arquitetura Modular)



```
1 def validar_dados(nome, idade):
2     if nome == "" or idade <= 18:
3         return False
4     return True
5
6 def calcular_bonus(salario):
7     if salario > 5000:
8         return salario * 0.10
9     return salario * 0.05
10
11 def cadastrar_funcionario(nome, idade, salario):
12     if validar_dados(nome, idade):
13         bonus = calcular_bonus(salario)
14         print(f"Funcionário {nome.title()} cadastrado.")
15         print(f"Bônus calculado: R$ {bonus}")
16     else:
17         print("Erro no cadastro.")
18
19 cadastrar_funcionario("felipe dias", 32, 6500)
20
```


ArquivoEditarSeleçãoVerAcessarExecutarTerminalAjuda

AulasPython [Administrador]

aulas.py

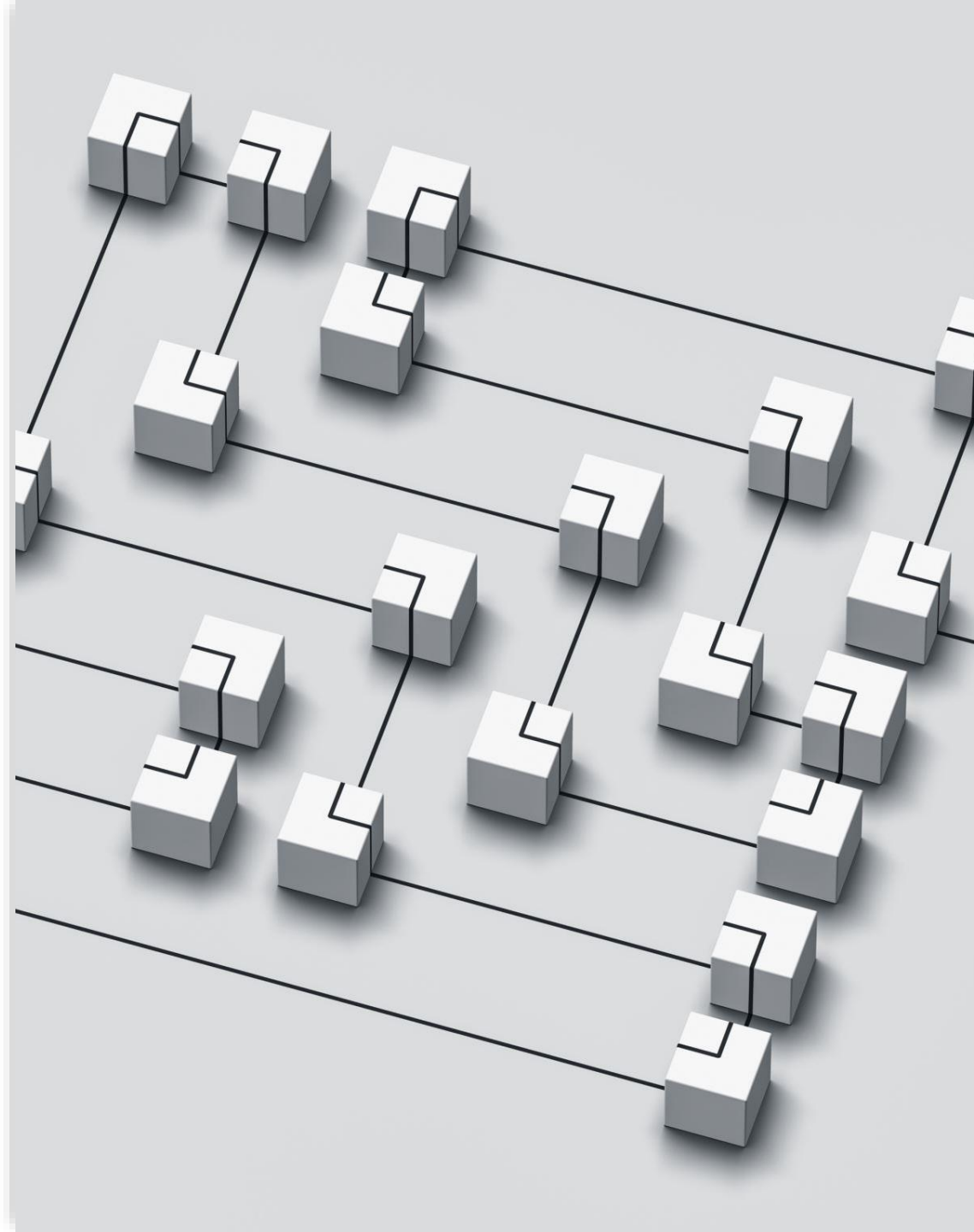
aulas.py

```
1 # Define função para validar nome e idade
2 def validar_dados(nome, idade):
3     # Verifica se o nome está vazio ou idade inválida
4     if nome == "" or idade <= 0:
5         # Retorna falso caso os dados sejam inválidos
6         return False
7     # Retorna verdadeiro se os dados estiverem corretos
8     return True
9
10 # Define função para calcular bônus baseado no salário
11 def calcular_bonus(salario):
12     # Verifica se o salário é maior que 5000
13     if salario > 5000:
14         # Retorna 10% de bônus
15         return salario * 0.10
16     # Caso contrário, retorna 5% de bônus
17     return salario * 0.05
18
19 # Define função principal de cadastro de funcionário
20 def cadastrar_funcionario(nome, idade, salario):
21     # Chama a função de validação antes de continuar
22     if validar_dados(nome, idade):
23         # Calcula o bônus chamando outra função
24         bonus = calcular_bonus(salario)
25         # Exibe mensagem de cadastro com nome formatado
26         print(f"Funcionário {nome.title()} cadastrado.")
27         # Exibe valor do bônus calculado
28         print(f"Bônus calculado: R$ {bonus}")
29     else:
30         # Exibe mensagem de erro caso dados sejam inválidos
31         print("Erro no cadastro.")
32
33 # Executa o cadastro com dados de exemplo
34 cadastrar_funcionario("felipe dias", 32, 6500)
```

Ln 34, Col 47Espaços: 4UTF-8CRLFPythonFinish Setup3.13.2

Análise técnica:

- Aplicação de múltiplas funções com responsabilidades distintas;
- Implementação do princípio da modularização;
- Uso de função como mecanismo de validação;
- Uso de função como mecanismo de cálculo;
- Função principal coordena chamadas internas;
- Estrutura semelhante a backend de sistema empresarial;
- Demonstra reutilização e organização profissional do código.

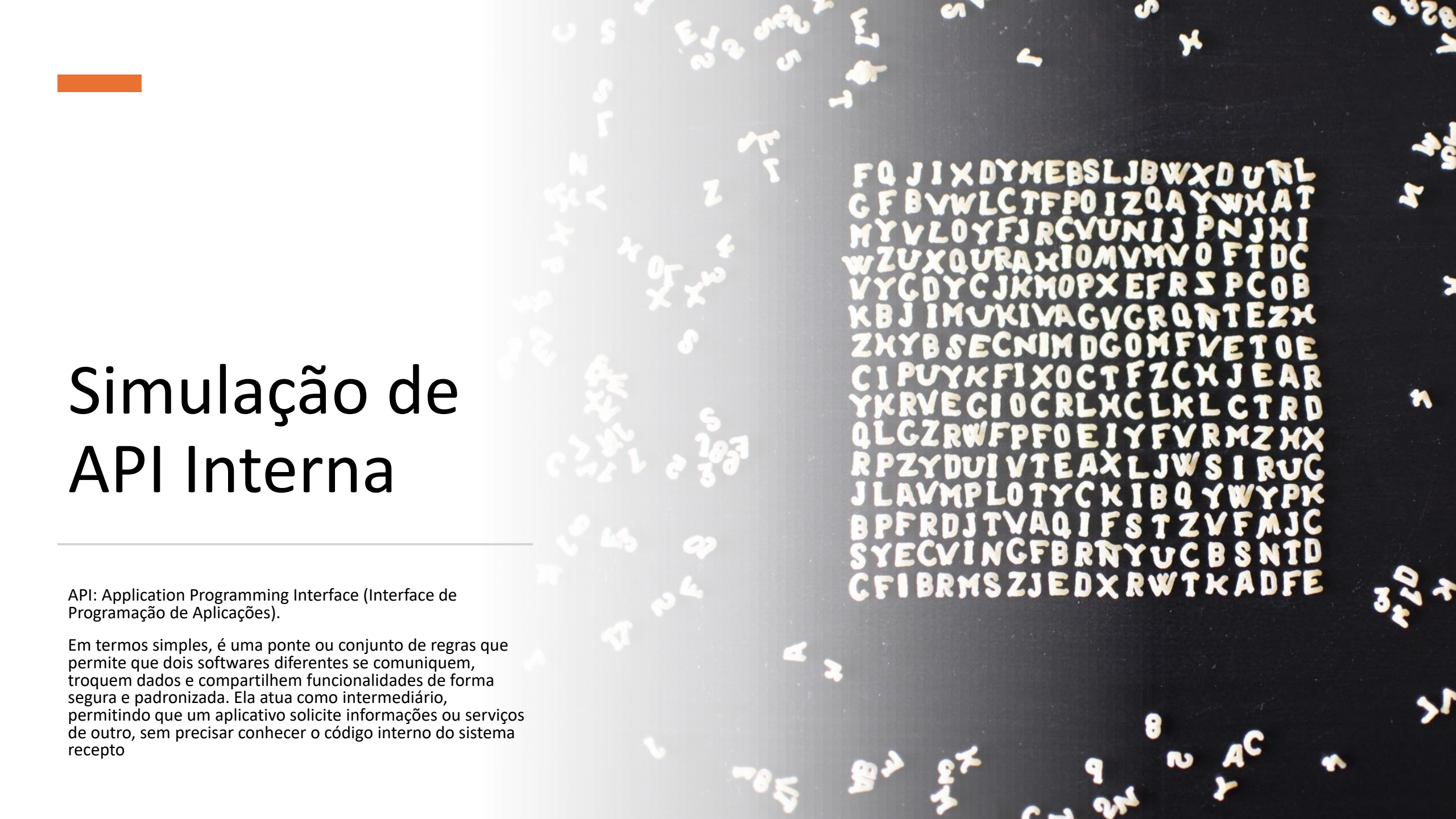




Simulação de API Interna

API: Application Programming Interface (Interface de Programação de Aplicações).

Em termos simples, é uma ponte ou conjunto de regras que permite que dois softwares diferentes se comuniquem, troquem dados e compartilhem funcionalidades de forma segura e padronizada. Ela atua como intermediário, permitindo que um aplicativo solicite informações ou serviços de outro, sem precisar conhecer o código interno do sistema recepto



FQ J I X D Y M E B S L J B W X D U N L
G F B V W L C T F P O I Z Q A Y W H A T
M Y V L O Y F J R C V U N I J P N J H I
W Z U X Q U R A X I O M V M V O F T D C
V Y C D Y C J K M O P X E F R S P C O B
K B J I M V K I V A G V G R Q N T E Z H
Z H Y B S E C N I M D G O M F V E T O E
C I P U Y K F I X O C T F Z C H J E A R
Y K R V E G I O C R L X C L K L C T R D
Q L G Z R W F P F O E I Y F V R M Z H X
R P Z Y D U I V T E A X L J W S I R U C
J L A V M P L O T Y C K I B Q Y W Y P K
B P F R D J T V A Q I F S T Z V F M J C
S Y E C V I N G F B R N Y U C B S N T D
C F I B R M S Z J E D X R W T K A D F E

ArquivoEditarSeleçãoVerAcessarExecutarTerminalAjuda

Q AulasPython

aulas.py x

aulas.py

1def gerar_token(usuario):

2 return f"TOKEN_{usuario.upper()}_2026"

3

4def registrar_log(usuario):

5 print(f"Log registrado para usuário {usuario}")

6

7def acessar_sistema(usuario):

8 token = gerar_token(usuario)

9 registrar_log(usuario)

10 return f"Acesso autorizado com {token}"

11

12 print(acessar_sistema("felipe dias"))

13

Ln 13, Col 1Espaços: 4UTF-8CRLFPythonFinish Setup


```
1 # Define função para gerar um token baseado no nome do usuário
2 def gerar_token(usuario):
3
4     # Retorna uma string formatada com o nome em maiúsculo
5     return f"TOKEN_{usuario.upper()}_2026"
6
7 # Define função para registrar log de acesso
8 def registrar_log(usuario):
9
10    # Exibe mensagem simulando registro de log no sistema
11    print(f"Log registrado para usuário {usuario}")
12
13 # Define função principal que simula acesso ao sistema
14 def acessar_sistema(usuario):
15
16    # Gera um token chamando a função gerar_token
17    token = gerar_token(usuario)
18
19    # Registra o log de acesso chamando a função registrar_log
20    registrar_log(usuario)
21
22    # Retorna mensagem final com o token gerado
23    return f"Acesso autorizado com {token}"
24
25 # Chama a função principal simulando acesso do usuário
26 print(acessar_sistema("felipe dias"))
27
```

Análise técnica:

- Estrutura dividida em camadas de responsabilidade;
- Uma função gera identificação (token);
- Outra registra evento (log);
- A função principal orquestra as execuções;
- Demonstra encadeamento funcional;
- Simula arquitetura de microserviço simplificada;
- Representa modelo comum em APIs REST e sistemas corporativos.

Obs: APIs REST (Representational State Transfer) é um estilo arquitetural para desenvolver serviços web que permite a comunicação entre sistemas através de protocolos HTTP, utilizando métodos padrão (GET, POST, PUT, DELETE) para gerenciar recursos, geralmente retornando dados em JSON. São sem estado (stateless), flexíveis e ideais para aplicações distribuídas.

